

Handbuch zur Nutzung von Napari für die Analyse von Spektraldaten

Einleitung

Napari ist eine Open-Source-Plattform für die wissenschaftliche Bildverarbeitung, die sich durch ihre Flexibilität und Erweiterbarkeit auszeichnet. Dieses Handbuch empfiehlt Napari als Werkzeug zur Analyse von Spektraldaten, wie sie von einem DIY Matchbox-Spektrometer geliefert werden. Napari eignet sich besonders für die Visualisierung, Bearbeitung und Auswertung von hyperspektralen Daten, die in Form von Bildwürfeln (Image Cubes) vorliegen.

1. Installation und Einrichtung

1.0 Hardware-Anforderungen für Echtzeit-Übertragung

- **ESP32-Board** (z. B. ESP32-CAM oder ESP32 mit OV2640-Kamera-Modul)
- **Kamera-Modul** (z. B. OV2640, 2MP)
- **Stabile Stromversorgung** (5V, mind. 2A für ESP32-CAM)
- **WLAN-Netzwerk** (2,4 GHz, da ESP32 kein 5 GHz unterstützt)
- **PC mit Napari-Server** (Python 3.9+, Napari installiert)

1.0.1 Schaltplan für ESP32-CAM

ESP32-CAM Pin	Verbindung
5V	Externe Stromversorgung (5V, 2A)
GND	GND
GPIO 0	Nicht verbinden (für Bootloader-Modus)
GPIO 2	Onboard-LED
GPIO 12	Kamera-Datenleitung (PCLK)
GPIO 13	Kamera-Datenleitung (HREF)
GPIO 14	Kamera-Datenleitung (VSYNC)
GPIO 15	Kamera-Datenleitung (XCLK)
GPIO 16	Kamera-Datenleitung (SIOD)
GPIO 17	Kamera-Datenleitung (SIOC)
GPIO 18	Kamera-Datenleitung (D0)
GPIO 19	Kamera-Datenleitung (D1)
GPIO 21	Kamera-Datenleitung (D2)
GPIO 22	Kamera-Datenleitung (D3)
GPIO 23	Kamera-Datenleitung (D4)
GPIO 25	Kamera-Datenleitung (D5)

ESP32-CAM Pin	Verbindung
GPIO 26	Kamera-Datenleitung (D6)
GPIO 27	Kamera-Datenleitung (D7)
GPIO 32	Kamera-Datenleitung (D8)
GPIO 33	Kamera-Datenleitung (D9)

Hinweis: Die ESP32-CAM hat einen integrierten OV2640-Sensor. Achten Sie auf eine stabile Stromversorgung, um Abstürze zu vermeiden.

1.0.2 Software-Anforderungen für ESP32

- **Arduino IDE** oder **PlatformIO** (für die Programmierung des ESP32)
- **ESP32-Board-Support** in Arduino IDE:
 - Öffnen Sie die Arduino IDE.
 - Gehen Sie zu **File > Preferences** und fügen Sie folgende URL unter **Additional Boards Manager URLs** hinzu:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

- Gehen Sie zu **Tools > Board > Boards Manager** und suchen Sie nach **esp32**. Installieren Sie das Paket.
- **Benötigte Bibliotheken:**
 - **WiFi** (vorinstalliert)
 - **HTTPClient** (vorinstalliert)
 - **esp_camera** (für Kamerafunktionen)
 - **ArduinoJson** (für JSON-Datenverarbeitung, optional)

Installation der Bibliotheken über **Sketch > Include Library > Manage Libraries**.

1. Installation und Einrichtung

1.1 Voraussetzungen

- Python 3.9 oder höher
- pip (Python-Paketmanager)
- Empfohlen: Virtuelle Umgebung (z. B. **venv** oder **conda**)

1.2 Installation von Napari

Napari kann über pip installiert werden:

```
pip install napari
```

Für die Arbeit mit Spektraldaten werden zusätzliche Plugins benötigt. Installieren Sie diese mit:

```
pip install napari-plugin-engine
pip install napari-spectral
```

1.3 Installation spezifischer Plugins für Spektraldaten

Für die Analyse von Spektraldaten werden folgende Plugins empfohlen:

- **napari-spectral**: Ermöglicht die Visualisierung und Analyse von hyperspektralen Daten.
- **napari-plot-profile**: zur Erstellung von Profilen und Spektren.
- **napari-skimage-regionprops**: zur Segmentierung und Analyse von Regionen.

Installation:

```
pip install napari-spectral napari-plot-profile napari-skimage-regionprops
```

1.4 Starten von Napari

Napari kann über die Kommandozeile gestartet werden:

```
napari
```

Alternativ kann Napari auch aus einem Python-Skript heraus gestartet werden:

```
import napari
napari.run()
```

2. Datenimport und Vorbereitung

2.0 Echtzeit-Übertragung von ESP32 zu Napari

Um Spektraldaten oder Bilder direkt vom ESP32 an Napari zu übertragen, gibt es zwei Ansätze:

1. **HTTP-Server auf dem PC**: Ein Python-Skript empfängt Bilder vom ESP32 und leitet sie an Napari weiter.
2. **MQTT-Protokoll**: Leichter und effizienter für Echtzeit-Datenströme.

Hier wird der **HTTP-Server-Ansatz** beschrieben, da er einfacher zu implementieren ist.

2.0.1 Python-Skript: HTTP-Server für Napari

Dieses Skript startet einen lokalen HTTP-Server, der Bilder vom ESP32 empfängt und sie direkt in Napari anzeigt.

```

from http.server import HTTPServer, SimpleHTTPRequestHandler
import threading
import napari
import numpy as np
from PIL import Image
import io
import cv2

class Esp32ImageHandler(SimpleHTTPRequestHandler):
    def __init__(self, *args, viewer=None, **kwargs):
        self.viewer = viewer
        super().__init__(*args, **kwargs)
    def do_POST(self):
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length)

        # Bilddaten verarbeiten
        image = Image.open(io.BytesIO(post_data))
        image_np = np.array(image)

        # Bild in Napari anzeigen
        if hasattr(self, 'viewer') and self.viewer is not None:
            self.viewer.add_image(image_np, name=f'ESP32_Bild_{threading.get_ident()}')

        # Antwort senden
        self.send_response(200)
        self.send_header('Content-type', 'text/plain')
        self.end_headers()
        self.wfile.write(b'Bild empfangen und in Napari angezeigt')
    def start_server(port=8000, viewer=None):
        server_address = ('', port)
        handler_class = lambda *args, **kwargs: Esp32ImageHandler(*args,
viewer=viewer, **kwargs)
        httpd = HTTPServer(server_address, handler_class)
        print(f'HTTP-Server läuft auf Port {port}')
        httpd.serve_forever()
    # Napari-Viewer starten
    viewer = napari.Viewer()
    # HTTP-Server in einem separaten Thread starten
    server_thread = threading.Thread(target=start_server, args=(8000, viewer),
daemon=True)
    server_thread.start()
    # Napari anzeigen
    napari.run()

```

Anleitung zum Ausführen des Servers:

1. Speichern Sie das Skript als `napari_http_server.py`.
2. Installieren Sie die benötigten Bibliotheken:

```
pip install numpy opencv-python pillow
```

3. Starten Sie das Skript:

```
python napari_http_server.py
```

4. Der Server läuft auf `http://<Ihre-IP>:8000`. Notieren Sie sich die IP-Adresse Ihres PCs.

2.0.2 ESP32-Code: Bilder erfassen und an den Server senden

Der folgende Code erfasst Bilder mit der ESP32-CAM und sendet sie an den HTTP-Server.

```
#include "WiFi.h"
#include "esp_camera.h"
#include "HTTPClient.h"
// WLAN-Anmeldeinformationen
const char* ssid = "Ihr_WLAN_SSID";
const char* password = "Ihr_WLAN_Passwort";
// Server-URL (IP-Adresse Ihres PCs)
const char* serverUrl = "http://192.168.1.100:8000";
// Kamera-Pins (ESP32-CAM)
#define CAMERA_MODEL_AI_THINKER
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22
void setup() {
    Serial.begin(115200);

    // Kamera initialisieren
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
```

```

config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_UXGA; // 1600x1200 (kann angepasst werden)
config.jpeg_quality = 12; // 10-63 (niedriger = bessere Qualität)
config.fb_count = 1;

// Kamera initialisieren
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Kamera-Initialisierung fehlgeschlagen: 0x%x", err);
    return;
}
// Verbindung zum WLAN herstellen
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("Verbunden mit WLAN");
Serial.print("IP-Adresse: ");
Serial.println(WiFi.localIP());
}

void loop() {
    // Bild erfassen
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("Fehler beim Erfassen des Bildes");
        return;
    }
    // HTTP-Client initialisieren
    HTTPClient http;
    http.begin(serverUrl);
    http.addHeader("Content-Type", "image/jpeg");

    // Bild an den Server senden
    int httpResponseCode = http.POST(fb->buf, fb->len);

    if (httpResponseCode > 0) {
        Serial.print("Bild gesendet. Server-Antwort: ");
        Serial.println(httpResponseCode);
    } else {
        Serial.print("Fehler beim Senden: ");
        Serial.println(httpResponseCode);
    }
}

```

```
// Ressourcen freigeben
http.end();
esp_camera_fb_return(fb);

// Wartezeit vor dem nächsten Bild
delay(5000); // 5 Sekunden Wartezeit
}
```

Anleitung zum Hochladen des Codes auf den ESP32:

1. Öffnen Sie die Arduino IDE.
2. Wählen Sie das richtige Board aus: **Tools > Board > ESP32 Arduino > AI Thinker ESP32-CAM**.
3. Wählen Sie den richtigen Port aus: **Tools > Port > /dev/ttyUSB0** (oder der entsprechende COM-Port unter Windows).
4. Kopieren Sie den obigen Code in die Arduino IDE.
5. Passen Sie die WLAN-Anmeldeinformationen (`ssid` und `password`) sowie die Server-URL (`serverUrl`) an.
6. Klicken Sie auf **Sketch > Upload**, um den Code auf den ESP32 hochzuladen.

Hinweis:

- Halten Sie die **BOOT-Taste** auf der ESP32-CAM gedrückt, während Sie auf **Upload** klicken, und lassen Sie sie los, sobald der Upload beginnt.
- Die ESP32-CAM hat keinen USB-Anschluss. Sie benötigen einen **USB-zu-Serial-Adapter** (z. B. FT232RL oder CP2102) zum Programmieren.

2.0.3 Alternative: MQTT für Echtzeit-Datenübertragung

Für eine effizientere Echtzeit-Übertragung können Sie das **MQTT-Protokoll** verwenden. Hier ist eine kurze Anleitung:

1. MQTT-Broker installieren (z. B. Mosquitto):

```
# Auf Ubuntu/Debian
sudo apt update
sudo apt install mosquitto mosquitto-clients
```

2. Python-Skript für MQTT-Empfänger:

```
import paho.mqtt.client as mqtt
import napari
import numpy as np
from PIL import Image
import io
def on_message(client, userdata, msg):
    try:
```

```

image_data = msg.payload
image = Image.open(io.BytesIO(image_data))
image_np = np.array(image)
viewer.add_image(image_np, name=f'MQTT_Bild_{np.random.randint(0, 1000)}')
except Exception as e:
    print(f"Fehler beim Verarbeiten des Bildes: {e}")
# Napari-Viewer starten
viewer = napari.Viewer()
# MQTT-Client einrichten
client = mqtt.Client()
client.on_message = on_message
client.connect("127.0.0.1", 1883, 60)
client.subscribe("esp32/camera")
# MQTT-Client in einem Thread starten
import threading
client.loop_start()
# Napari anzeigen
napari.run()

```

3. ESP32-Code für MQTT-Sender:

```

#include "WiFi.h"
#include "esp_camera.h"
#include "PubSubClient.h"
// WLAN-Anmeldeinformationen
const char* ssid = "Ihr_WLAN_SSID";
const char* password = "Ihr_WLAN_Passwort";
// MQTT-Broker-Einstellungen
const char* mqtt_server = "192.168.1.100"; // IP-Adresse Ihres MQTT-Brokers
const char* topic = "esp32/camera";
WiFiClient espClient;
PubSubClient mqttClient(espClient);
// Kamera-Pins (wie im vorherigen Beispiel)
#define CAMERA_MODEL_AI_THINKER
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22
void setup() {
    Serial.begin(115200);

```



```

// Kamera initialisieren (wie im vorherigen Beispiel)
camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_UXGA;
config.jpeg_quality = 12;
config.fb_count = 1;

esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Kamera-Initialisierung fehlgeschlagen: 0x%x", err);
    return;
}

// Verbindung zum WLAN herstellen
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("Verbunden mit WLAN");

// MQTT-Client konfigurieren
mqttClient.setServer(mqtt_server, 1883);
}

void reconnect() {
    while (!mqttClient.connected()) {
        Serial.print("Versuche, Verbindung zum MQTT-Broker herzustellen...");
        String clientId = "ESP32-CAM-" + String(WiFi.macAddress());
        if (mqttClient.connect(clientId.c_str())) {
            Serial.println("Verbunden mit MQTT-Broker");
        } else {
            Serial.print("Fehlgeschlagen, rc=");
            Serial.print(mqttClient.state());
            Serial.println(" Versuche es in 5 Sekunden erneut...");
        }
    }
}

```

```

    delay(5000);
}
}
}
void loop() {
    if (!mqttClient.connected()) {
        reconnect();
    }
    mqttClient.loop();

    // Bild erfassen
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("Fehler beim Erfassen des Bildes");
        return;
    }
    // Bild über MQTT senden
    mqttClient.publish(topic, fb->buf, fb->len);

    // Ressourcen freigeben
    esp_camera_fb_return(fb);

    // Wartezeit vor dem nächsten Bild
    delay(5000);
}

```

4. Bibliotheken für MQTT installieren:

1. Installieren Sie die **PubSubClient**-Bibliothek in der Arduino IDE:
 - Gehen Sie zu **Sketch > Include Library > Manage Libraries**.
 - Suchen Sie nach **PubSubClient** und installieren Sie sie.
2. Installieren Sie die **paho-mqtt**-Bibliothek für Python:

```
pip install paho-mqtt
```

2. Datenimport und Vorbereitung

2.1 Datenformat des DIY Matchbox-Spektrometers

Ein DIY Matchbox-Spektrometer liefert typischerweise Spektraldaten in folgenden Formaten:

- **CSV-Dateien:** Enthalten Wellenlängen und zugehörige Intensitätswerte.
- **Bilddateien:** Hyperspektrale Würfel als mehrdimensionale Arrays (z. B. `.npy`, `.tif`).
- **Textdateien:** Rohdaten, die in ein lesbares Format umgewandelt werden müssen.

2.2 Import von Spektraldaten in Napari

Option 1: Import von Bildwürfeln (Image Cubes)

1. Öffnen Sie Napari.
2. Klicken Sie auf **File > Open** oder ziehen Sie die Datei per Drag & Drop in den Viewer.
3. Unterstützte Formate: `.tif`, `.npy`, `.hdf5`.

Option 2: Import von CSV-Daten

1. Konvertieren Sie die CSV-Datei in ein für Napari lesbares Format (z. B. `.npy`). Beispiel mit Python:

```
import numpy as np
import pandas as pd
# CSV-Datei einlesen
data = pd.read_csv('spektren.csv')
# In ein 3D-Array umwandeln (Beispiel: 100x100x256 für 100x100 Pixel und 256
Wellenlängen)
spectral_cube = np.random.rand(100, 100, 256) # Ersetzen Sie dies durch Ihre
Daten
# Speichern als .npy-Datei
np.save('spectral_cube.npy', spectral_cube)
```

2. Laden Sie die `.npy`-Datei in Napari.

Option 3: Direkter Import über Python-Skript

```
import napari
import numpy as np
# Spektraldaten laden (Beispiel: 100x100x256)
data = np.load('spektren.npy')
# Napari-Viewer starten
viewer = napari.Viewer()
viewer.add_image(data, name='Spektralwürfel')
napari.run()
```

3. Visualisierung von Spektraldaten

3.1 Grundlegende Visualisierung

1. Nach dem Import wird der Spektralwürfel als 3D-Datensatz angezeigt.
2. Nutzen Sie die **Layer Controls** (rechts im Fenster), um die Darstellung anzupassen:
 - **Colormap**: Wählen Sie eine Farbskala (z. B. `viridis`, `inferno`).
 - **Contrast**: Passen Sie die Helligkeit und den Kontrast an.
 - **Opacity**: Regulieren Sie die Transparenz.

3.2 Verwendung des napari-spectral Plugins

1. Installieren Sie das Plugin (falls noch nicht geschehen, siehe Abschnitt 1.3).
2. Öffnen Sie den Spektralwürfel in Napari.
3. Nutzen Sie die Werkzeuge des Plugins:
 - **Spectrum Viewer**: Zeigt das Spektrum für einen ausgewählten Pixel an.
 - **Wavelength Slider**: Ermöglicht das Durchblättern der Wellenlängen.
 - **Band Selection**: Wählen Sie einzelne spektrale Bänder zur Analyse aus.

Beispiel: Spektrum eines Pixels anzeigen

1. Klicken Sie auf einen Pixel im Spektralwürfel.
2. Öffnen Sie den **Spectrum Viewer** (über das Plugin-Menü oder die Symbolleiste).
3. Das Spektrum für den ausgewählten Pixel wird angezeigt.

3.3 Erstellen von Spektralprofilen

Mit dem **napari-plot-profile** Plugin können Sie Spektralprofile erstellen:

1. Wählen Sie einen Bereich (z. B. eine Linie oder ein Rechteck) im Spektralwürfel aus.
2. Öffnen Sie das **Plot Profile** Werkzeug.
3. Das Plugin zeigt die Intensitätswerte entlang der ausgewählten Linie oder Region an.

4. Datenanalyse

4.1 Spektrale Merkmale extrahieren

Absorptionsbanden identifizieren

1. Nutzen Sie den **Spectrum Viewer**, um Spektren zu analysieren.
2. Suchen Sie nach typischen Absorptionsbanden (z. B. bei 670 nm für Chlorophyll).
3. Markieren Sie die Wellenlängenbereiche mit niedriger Intensität.

Beispiel: Absorptionsbande bei 670 nm

- Wählen Sie einen Pixel mit bekannter Vegetation aus.
- Analysieren Sie das Spektrum im **Spectrum Viewer**.
- Identifizieren Sie die Absorptionsbande bei ~670 nm (Chlorophyll-Absorption).

4.2 Spektrale Indizes berechnen

Spektrale Indizes wie der **NDVI (Normalized Difference Vegetation Index)** können direkt in Napari berechnet werden:

1. Verwenden Sie ein Python-Skript, um den Index zu berechnen:

```
import numpy as np
# Beispiel: NDVI berechnen (Band 80 = NIR, Band 40 = Rot)
band_nir = data[:, :, 80] # Nahinfrarot-Band
band_red = data[:, :, 40] # Rotes Band
```

```
ndvi = (band_nir - band_red) / (band_nir + band_red)
# NDVI in Napari anzeigen
viewer.add_image(ndvi, name='NDVI')
```

2. Fügen Sie das Ergebnis als neue Ebene in Napari hinzu.

4.3 Klassifizierung von Spektraldaten

Für die Klassifizierung von Spektraldaten können Sie:

- **Manuelle Klassifizierung:** Nutzen Sie die Segmentierungswerkzeuge von Napari, um Regionen zu markieren.
- **Automatische Klassifizierung:** Verwenden Sie Maschinenlernmethoden (z. B. `scikit-learn`).

Beispiel: Einfache Klassifizierung mit scikit-learn

```
from sklearn.cluster import KMeans
import numpy as np
# Daten vorbereiten (Beispiel: 10000 Pixel, 256 Bänder)
pixels = data.reshape(-1, 256)
# K-Means Klassifizierung
kmeans = KMeans(n_clusters=3)
labels = kmeans.fit_predict(pixels)
# Labels in die ursprüngliche Form zurückbringen
labels = labels.reshape(data.shape[:2])
# In Napari anzeigen
viewer.add_labels(labels, name='Klassifizierung')
```

5. Export der Ergebnisse

5.1 Export von Bildern

1. Klicken Sie mit der rechten Maustaste auf die Ebene, die Sie exportieren möchten.
2. Wählen Sie **Save Layer As**.
3. Wählen Sie das gewünschte Format (z. B. `.tif`, `.png`).

5.2 Export von Spektraldaten

Export als CSV

```
import pandas as pd
# Spektrum eines Pixels exportieren
pixel_spectrum = data[50, 50, :] # Beispiel: Pixel bei (50, 50)
wavelengths = np.arange(400, 700, 1) # Beispiel: Wellenlängen von 400-700 nm
# In DataFrame umwandeln
df = pd.DataFrame({
    'Wellenlänge (nm)': wavelengths,
    'Intensität': pixel_spectrum
})
```

```

}))
# Als CSV speichern
df.to_csv('pixel_spectrum.csv', index=False)

```

Export als .npy

```

# Spektralwürfel exportieren
np.save('spectral_cube_processed.npy', data)

```

6. Tipps und Tricks

6.1 Performance-Optimierung

- **Daten reduzieren:** Arbeiten Sie mit Ausschnitten der Daten, um die Performance zu verbessern.

```

# Ausschnitt der Daten laden
subset = data[10:50, 10:50, :]
viewer.add_image(subset, name='Ausschnitt')

```

- **Downsampling:** Reduzieren Sie die Auflösung der Daten.

```

from skimage.transform import resize
# Daten herunterskalieren
downsampled = resize(data, (data.shape[0]//2, data.shape[1]//2, data.shape[2]))
viewer.add_image(downsampled, name='Herunterskaliert')

```

6.2 Nützliche Plugins

Plugin	Zweck
napari-spectral	Visualisierung und Analyse von hyperspektralen Daten
napari-plot-profile	Erstellen von Profilen und Spektren
napari-skimage-regionprops	Segmentierung und Analyse von Regionen
napari-segment-blobs	Automatische Segmentierung von Objekten
napari-animation	Erstellen von Animationen aus Spektraldaten

6.3 Fehlerbehebung

Problem	Lösung
Daten werden nicht geladen	Überprüfen Sie das Dateiformat. Konvertieren Sie ggf. in <code>.npy</code> oder <code>.tif</code> .
Langsame Performance	Reduzieren Sie die Datengröße oder verwenden Sie Ausschnitte.
Plugin funktioniert nicht	Überprüfen Sie die Installation: <code>pip install --upgrade <plugin-name></code> .
Spektrum wird nicht angezeigt	Stellen Sie sicher, dass der Spektralwürfel korrekt geladen wurde.

7. Beispiel-Workflow

Schritt-für-Schritt-Anleitung: Analyse eines Spektralwürfels

1. Daten laden

- Speichern Sie die Daten des Matchbox-Spektrometers als `.npy`-Datei.
- Laden Sie die Datei in Napari.

2. Daten visualisieren

- Passen Sie Colormap und Kontrast an.
- Nutzen Sie den **Wavelength Slider**, um durch die Wellenlängen zu blättern.

3. Spektrum analysieren

- Wählen Sie einen Pixel aus und öffnen Sie den **Spectrum Viewer**.
- Identifizieren Sie Absorptionsbanden.

4. Spektrale Indizes berechnen

- Berechnen Sie z. B. den NDVI (siehe Abschnitt 4.2).
- Fügen Sie das Ergebnis als neue Ebene hinzu.

5. Klassifizierung durchführen

- Segmentieren Sie die Daten manuell oder mit K-Means.
- Visualisieren Sie die Klassifizierungsergebnisse.

6. Ergebnisse exportieren

- Exportieren Sie die bearbeiteten Daten als `.tif` oder `.npy`.
- Exportieren Sie Spektren als CSV.

8. Ressourcen und Weiterführende Links

- [Napari Offizielle Dokumentation](#)
- [napari-spectral Plugin](#)
- [napari-plot-profile Plugin](#)
- [DIY Matchbox-Spektrometer \(Public Lab\)](#)
- [Scikit-learn für Klassifizierung](#)

9. Glossar

Begriff	Erklärung
Hyperspektral	Daten mit vielen spektralen Bändern (typischerweise > 100).
Spektralwürfel	3D-Datensatz mit räumlichen (x, y) und spektralen (λ) Dimensionen.
Absorptionsbande	Wellenlängenbereich, in dem Licht absorbiert wird (z. B. durch Chlorophyll).

Begriff	Erklärung
NDVI	Normalized Difference Vegetation Index, ein Maß für Vegetationsdichte.
Pixel-Spektrum	Spektrum eines einzelnen Pixels im Spektralwürfel.

10. Anhang: Python-Skript für den Komplett-Workflow

10.1 Fehlerbehebung für ESP32 und Kamera

Problem	Mögliche Ursache	Lösung
ESP32 wird nicht erkannt	Falscher Port oder Treiber fehlend	Installieren Sie den CP2102- oder FT232RL-Treiber. Überprüfen Sie den Port in der Arduino IDE.
Kamera liefert keine Bilder	Falsche Pin-Konfiguration	Überprüfen Sie die Kamera-Pins in der <code>camera_config_t</code> -Struktur.
Bilder sind schwarz oder verzerrt	Falsche Auflösung oder JPEG-Qualität	Passen Sie <code>frame_size</code> und <code>jpeg_quality</code> an (z. B. <code>FRAME_SIZE_SVGA</code> , <code>jpeg_quality = 10</code>).
Verbindungsabbrüche zum Server	Instabile WLAN-Verbindung	Verwenden Sie ein 2,4-GHz-WLAN-Netzwerk. Reduzieren Sie die Bildgröße oder erhöhen Sie die Wartezeit.
ESP32 stürzt ab	Strommangel	Verwenden Sie eine stabile 5V/2A-Stromversorgung.
HTTP-Server empfängt keine Daten	Falsche IP-Adresse oder Port	Überprüfen Sie die IP-Adresse des PCs und stellen Sie sicher, dass der Port (z. B. 8000) offen ist.
MQTT-Verbindung fehlgeschlagen	Falsche Broker-IP oder Port	Überprüfen Sie die IP-Adresse des MQTT-Brokers und den Port (Standard: 1883).

10.2 Optimierung der Bildübertragung

1. Bildgröße reduzieren

Die ESP32-CAM unterstützt verschiedene Auflösungen. Verwenden Sie eine kleinere Auflösung, um die Übertragungsgeschwindigkeit zu erhöhen:

```
// In der camera_config_t-Struktur
config.frame_size = FRAME_SIZE_QVGA; // 320x240
// oder
config.frame_size = FRAME_SIZE_VGA; // 640x480
```

2. JPEG-Qualität anpassen

Eine niedrigere JPEG-Qualität reduziert die Dateigröße:

```
config.jpeg_quality = 10; // 10-63 (niedriger = bessere Komprimierung)
```


3. Bildausschnitt (ROI) verwenden

Wenn nur ein bestimmter Bereich des Bildes relevant ist, können Sie einen Ausschnitt (Region of Interest, ROI) definieren:

```
sensor_t *s = esp_camera_sensor_get();  
s->set_vflip(s, 1); // Vertikal spiegeln (falls nötig)  
s->set_hmirror(s, 1); // Horizontal spiegeln (falls nötig)  
s->set_brightness(s, 1); // Helligkeit anpassen (-2 bis 2)  
s->set_contrast(s, 1); // Kontrast anpassen (-2 bis 2)
```

4. Wartezeit zwischen den Bildern anpassen

Reduzieren oder erhöhen Sie die Wartezeit im `loop()`:

```
delay(1000); // 1 Sekunde Wartezeit für schnellere Übertragung  
delay(10000); // 10 Sekunden Wartezeit für langsamere Übertragung
```

10.3 Sicherstellen der WLAN-Stabilität

1. Statische IP-Adresse für den ESP32

Um sicherzustellen, dass der ESP32 immer dieselbe IP-Adresse erhält, können Sie eine statische IP in Ihrem Router reservieren oder im ESP32-Code festlegen:

```
#include <WiFi.h>  
void setup() {  
    // ... (Kamera-Initialisierung)  
  
    // Statische IP-Adresse konfigurieren  
    IPAddress local_IP(192, 168, 1, 200); // Wählen Sie eine freie IP in Ihrem  
    Netzwerk  
    IPAddress gateway(192, 168, 1, 1); // IP Ihres Routers  
    IPAddress subnet(255, 255, 255, 0); // Subnetzmaske  
  
    if (!WiFi.config(local_IP, gateway, subnet)) {  
        Serial.println("Fehler beim Konfigurieren der statischen IP");  
    }  
  
    WiFi.begin(ssid, password);  
    // ... (Rest des Codes)  
}
```

2. WLAN-Signalstärke prüfen

Fügen Sie folgenden Code hinzu, um die WLAN-Signalstärke zu überwachen:

```

void loop() {
  // ... (Bild erfassen und senden)

  // WLAN-Signalstärke anzeigen
  long rssi = WiFi.RSSI();
  Serial.print("WLAN-Signalstärke (RSSI): ");
  Serial.print(rssi);
  Serial.println(" dBm");

  delay(5000);
}

```

Hinweis: Ein RSSI-Wert unter -70 dBm kann zu instabilen Verbindungen führen. Platzieren Sie den ESP32 näher am Router oder verwenden Sie einen WLAN-Repeater.

```

import napari
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
# 1. Daten laden (Beispiel: 100x100x256)
data = np.load('spektren.npy')
# 2. Napari-Viewer starten
viewer = napari.Viewer()
viewer.add_image(data, name='Spektralwürfel')
# 3. NDVI berechnen (Band 80 = NIR, Band 40 = Rot)
band_nir = data[:, :, 80]
band_red = data[:, :, 40]
ndvi = (band_nir - band_red) / (band_nir + band_red)
viewer.add_image(ndvi, name='NDVI')
# 4. Klassifizierung mit K-Means
pixels = data.reshape(-1, 256)
kmeans = KMeans(n_clusters=3)
labels = kmeans.fit_predict(pixels)
labels = labels.reshape(data.shape[:2])
viewer.add_labels(labels, name='Klassifizierung')
# 5. Spektrum eines Pixels exportieren
pixel_spectrum = data[50, 50, :]
wavelengths = np.arange(400, 700, 1)
df = pd.DataFrame({
    'Wellenlänge (nm)': wavelengths,
    'Intensität': pixel_spectrum
})
df.to_csv('pixel_spectrum.csv', index=False)
# Napari starten
napari.run()

```

Handbuch erstellt für die Nutzung von Napari zur Analyse von Spektraldaten eines DIY Matchbox-Spektrometers. Enthält zudem die Echtzeit-Übertragung von Bildern eines ESP32 mit Kamera an Napari.